When you highlight a chunk of text in an AI chatbot's response and click reply, it is **not** just a plain user message, nor is it a malicious security exploit. Instead, it relies on a **structured context injection** at the application layer.

The frontend UI and backend orchestrator package your reply alongside the specific snippet you referenced so the LLM understands *what* you are talking about.

How It Works Under the Hood

When you submit a quote-reply, the web app constructs the final message payload sent to the LLM API using one of two primary architectural patterns:

1. The Markdown Blockquote Pattern (Most Common)

The client-side app wraps your selected text into a standard Markdown blockquote, prepends it to your typed text, and appends the whole thing as a standard user message in the conversation history array.

- **What the LLM actually receives in the API payload:**
-

JSON

"role" "assistant" "content" "To implement authentication, you can use JSON Web Tokens (JWT) or session cookies. JWTs are stateless, whereas cookies require server-side session stores."
"role" "user" "content" "> JWTs are stateless\n\nCan you explain how token revocation works with them since they are stateless?"

-
- **Why it works:** Because LLMs are trained heavily on internet text (forums, GitHub issues, Slack threads) where quoting previous text using `>` or block formats is standard, the model natively understands that the quoted text is the focal point of your follow-up.
-

2. The Structured XML / JSON Tag Pattern (Advanced UIs)

Some advanced open-source UIs or proprietary enterprise wrappers use structured tags (like XML tags or JSON objects) behind the scenes to structurally bind your reply to the target chunk. This prevents ambiguity if the conversation is long.

- **What the LLM actually receives:**
-

XML

referenced_context

referenced_context
user_query

user_query

[redacted]

-
- **Why it's used:** It acts as a mild, deterministic form of prompt engineering. It explicitly forces the model's attention mechanism to lock onto the enclosed text block, eliminating the risk that the model thinks the quote is something *you* wrote.
-

Open-Source Projects & Platforms Implementing This

If you want to look at how developers implement this feature in open-source chat user interfaces, check out these repositories:

- open-webui / open-webui **(GitHub):** The most popular open-source UI for local LLMs (Ollama, OpenAI-compatible APIs). Its chat interface includes message-action components that handle quoting text chunks, formatting them into Markdown blockquotes, and threading them into the chat input box.
-
- lobehub / lobe-chat **(GitHub):** A high-polish, modern open-source UI framework. It features granular message interactions, including sub-menu selections on AI chat bubbles that pass selected text ranges directly into the chat composer as contextual references.
-
- danny-avila / LibreChat **(GitHub):** A comprehensive, multi-provider chat UI. It manages complex message trees and handles selective message/chunk referencing by appending structured metadata to the payload history.